整合性架構與設計文件 (Unified Architecture & Design Document) - AI 虛擬教師培訓平台

文件版本 (Document Version): `v1.0`
最後更新 (Last Updated): `2026-01-20`
主要作者 (Lead Author): 技術架構師
審核者 (Reviewers): 李耕竹（組長）、核心開發團隊
狀態 (Status): 草稿（Draft）

目錄 (Table of Contents)

- 第 1 部分：架構總覽 (Architecture Overview)
  - 1.1 C4 模型：視覺化架構
  - 1.2 DDD 戰略設計 (Strategic DDD)
  - 1.3 Clean Architecture 分層
  - 1.4 技術選型與決策
- 第 2 部分：需求摘要 (Requirements Summary)
- 第 3 部分：高層次架構設計 (High-Level Architectural Design)
- 第 4 部分：技術選型詳述 (Technology Stack Details)
- 第 5 部分：數據架構 (Data Architecture)
- 第 6 部分：部署與基礎設施架構 (Deployment and Infrastructure)
- 第 7 部分：跨領域考量 (Cross-Cutting Concerns)
- 第 8 部分：風險與緩解策略 (Risks and Mitigation)
- 第 9 部分：架構演進路線圖 (Architecture Evolution Roadmap)
- 第 10 部分：詳細設計 (Detailed Design)

目的: 本文件旨在將「AI 虛擬教師培訓平台」的業務需求轉化為一個完整、內聚的技術藍圖。系統透過 AI 模擬學生情境，協助教師練習薩提爾（Satir）溝通模式、社會情緒學習（SEL）框架等教育方法論，並提供專業評估回饋。

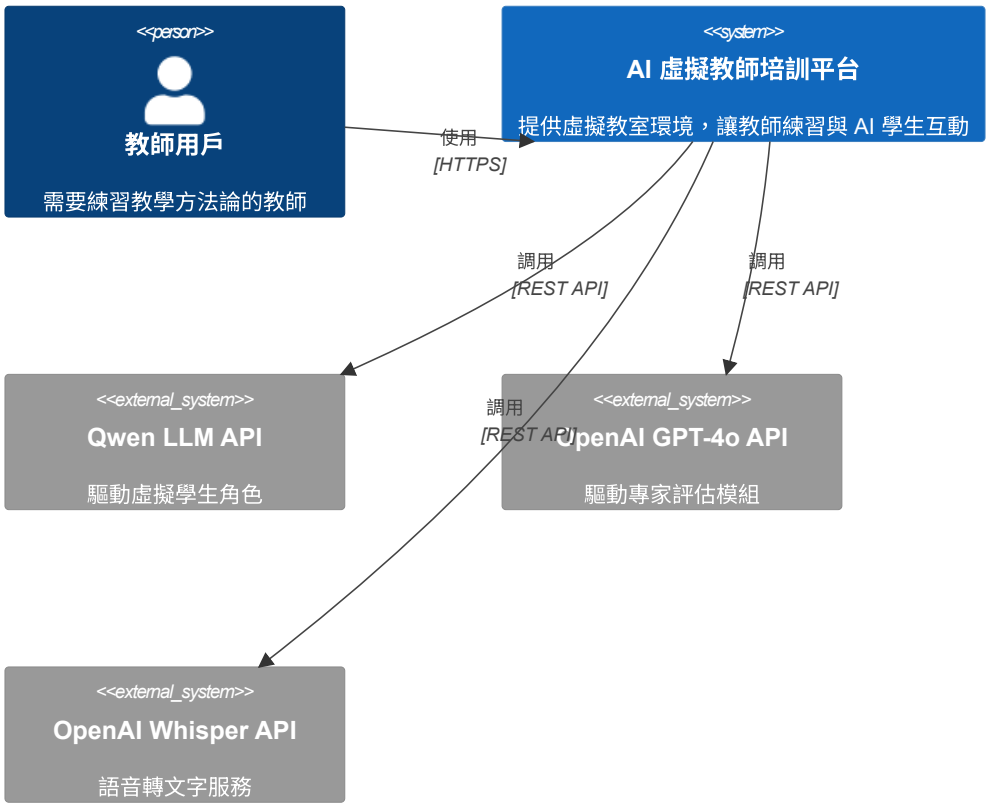
第 1 部分：架構總覽 (Architecture Overview)

此部分關注系統的宏觀結構與指導原則，回答「系統由什麼組成？」以及「它們之間如何互動？」。

1.1 C4 模型：視覺化架構

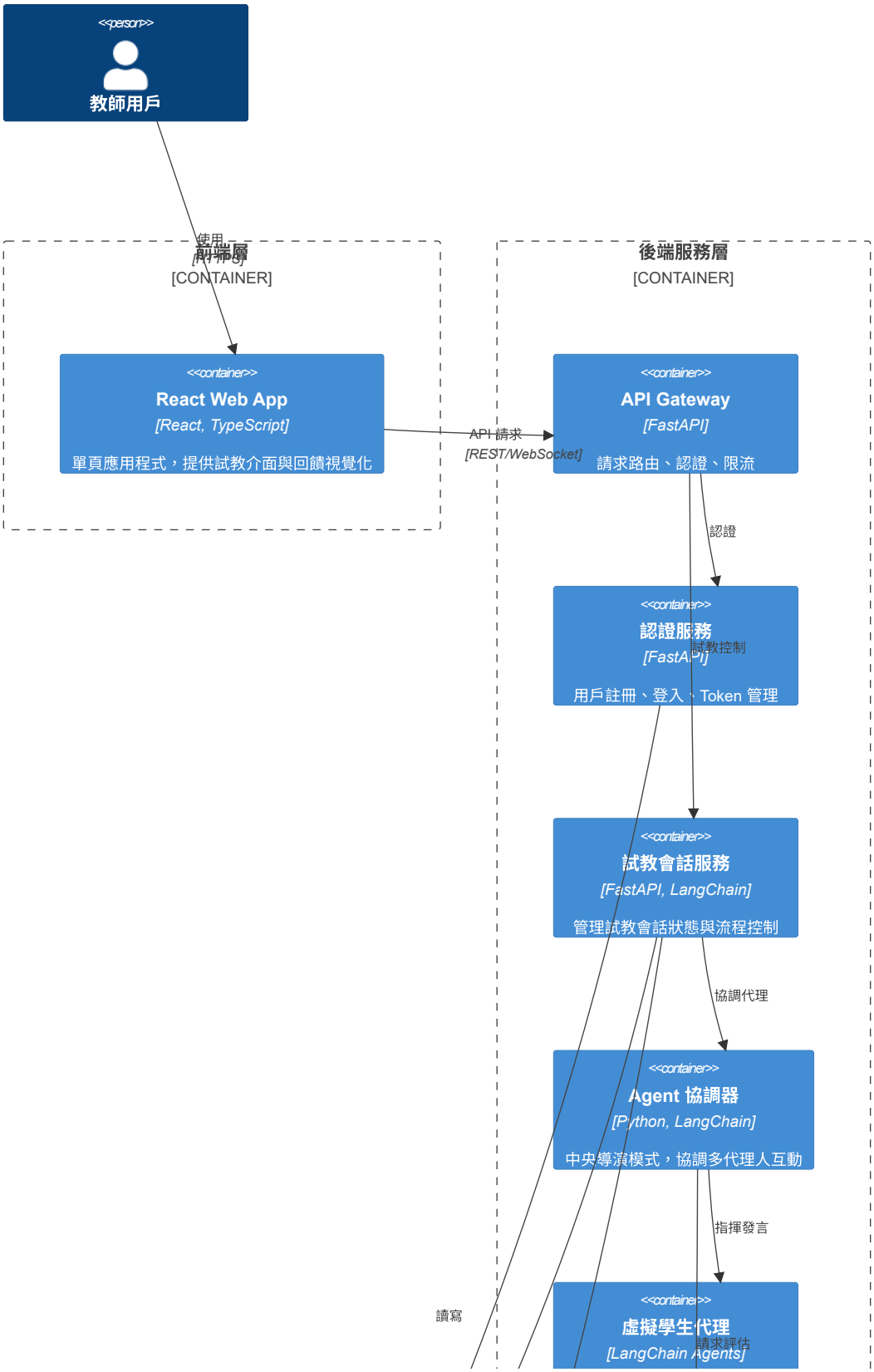
L1 - 系統情境圖 (System Context Diagram)

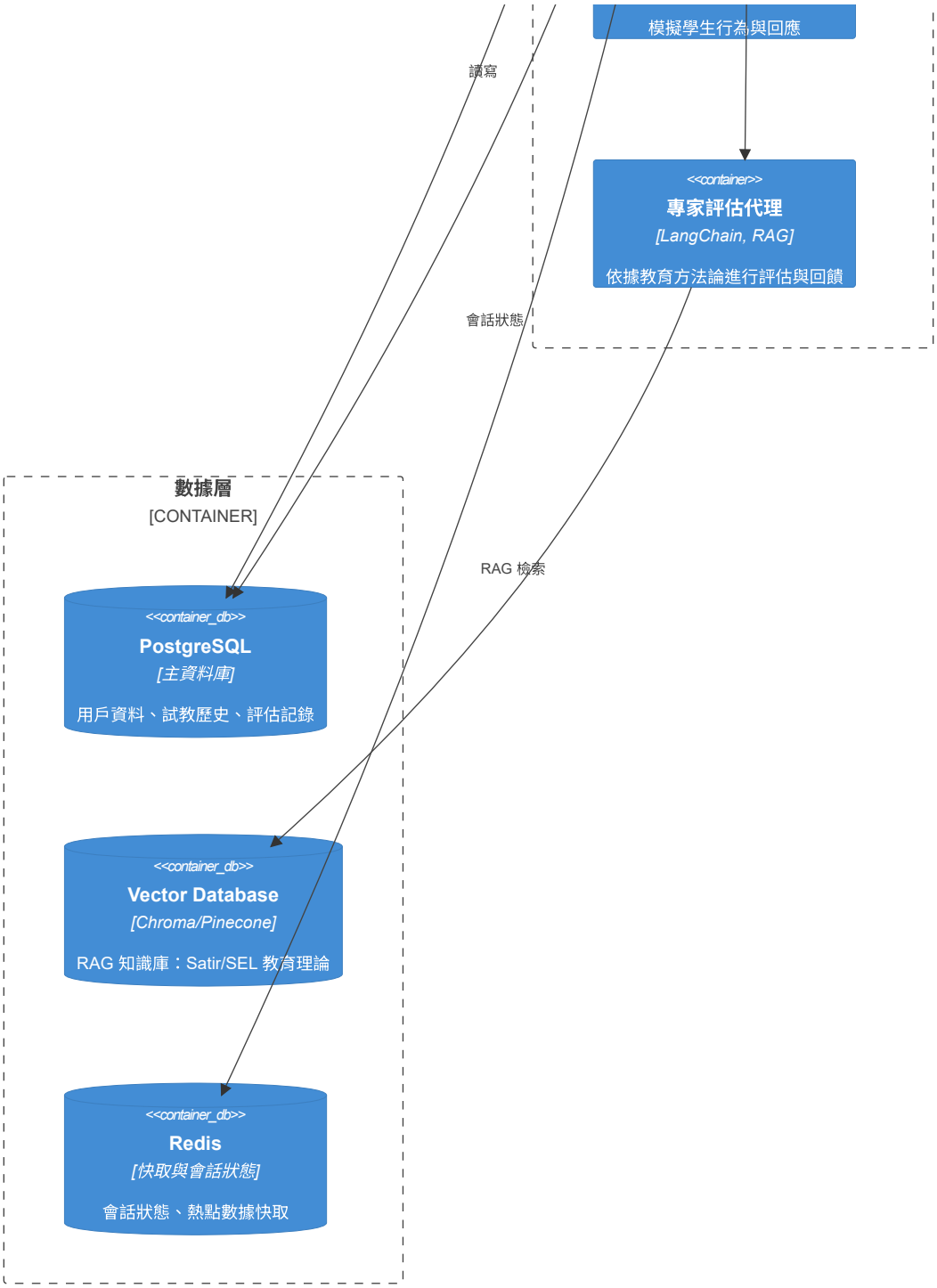
系統情境圖 - AI 虛擬教師培訓平台



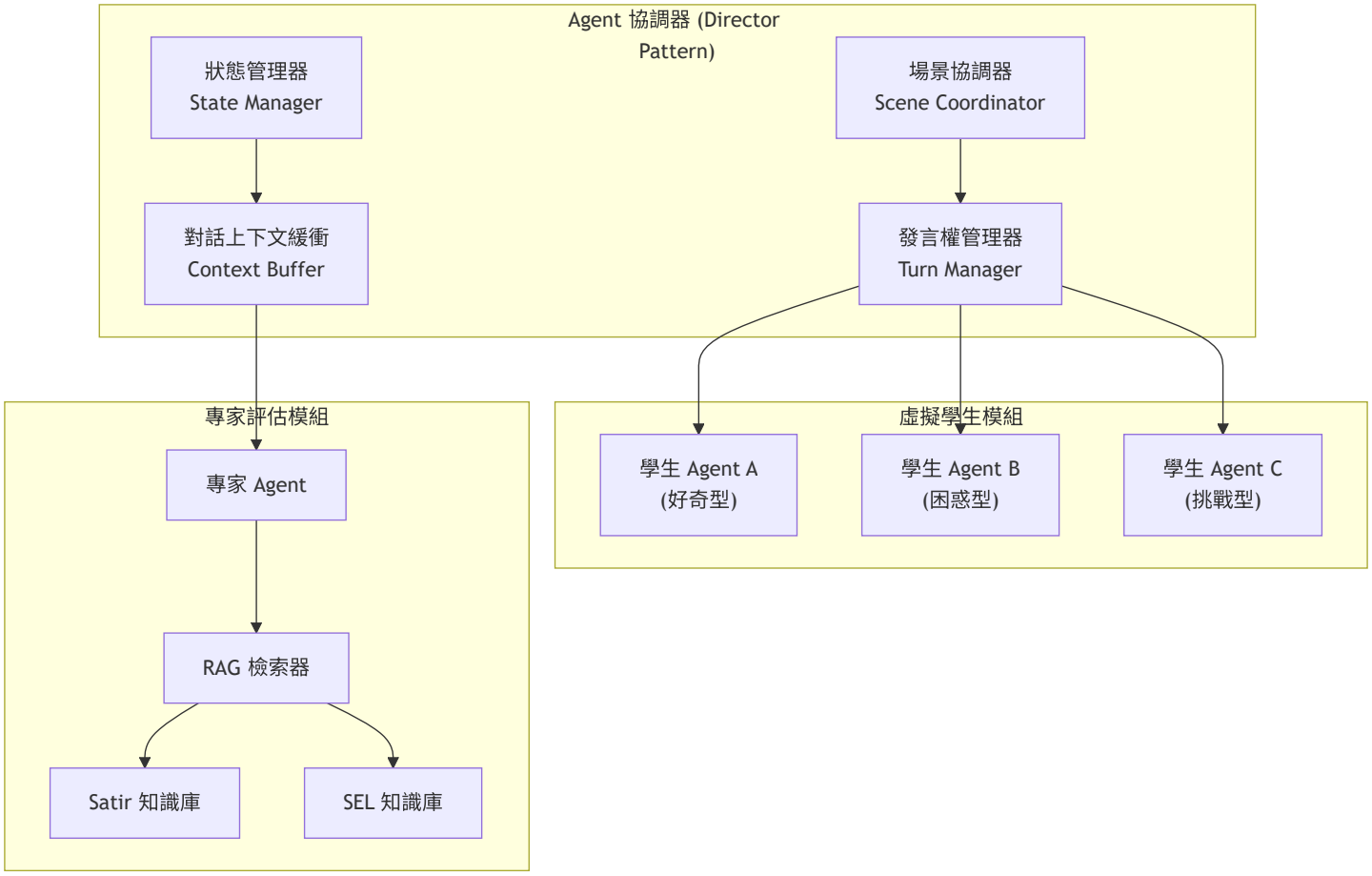
L2 - 容器圖 (Container Diagram)

容器圖 - AI 虚擬教師培訓平台





L3 - 元件圖 (Component Diagram) - Agent 協調器

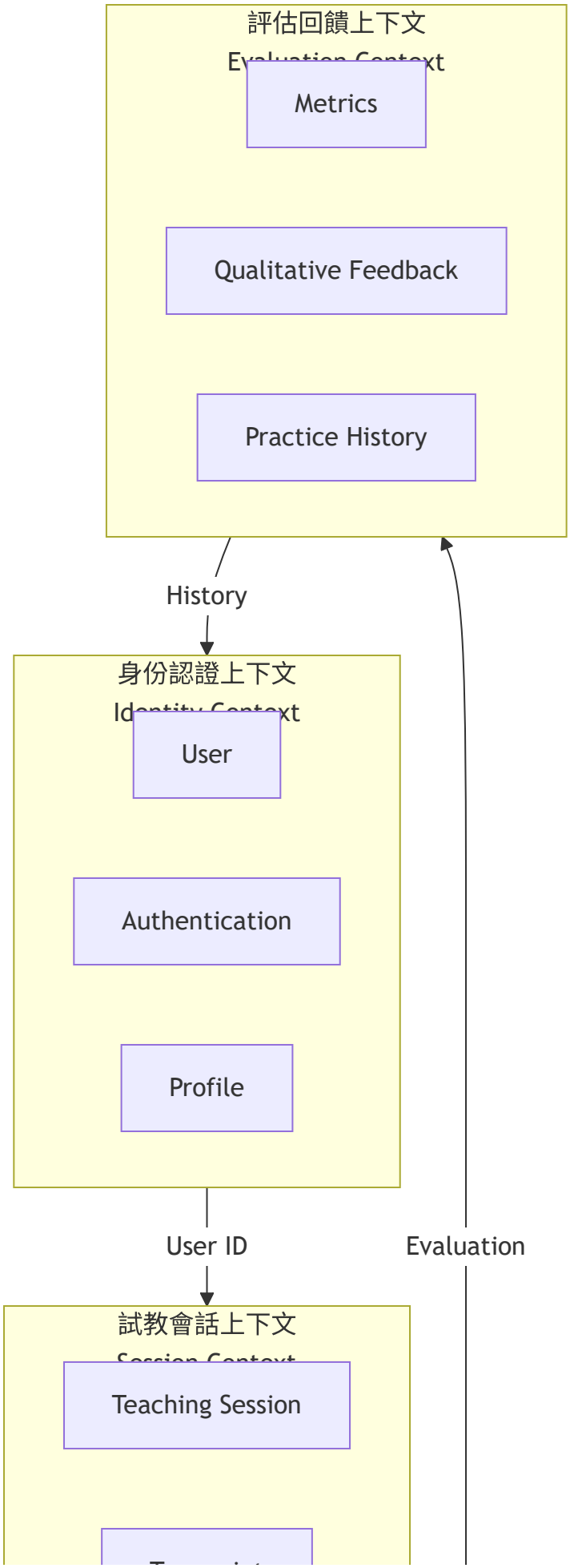


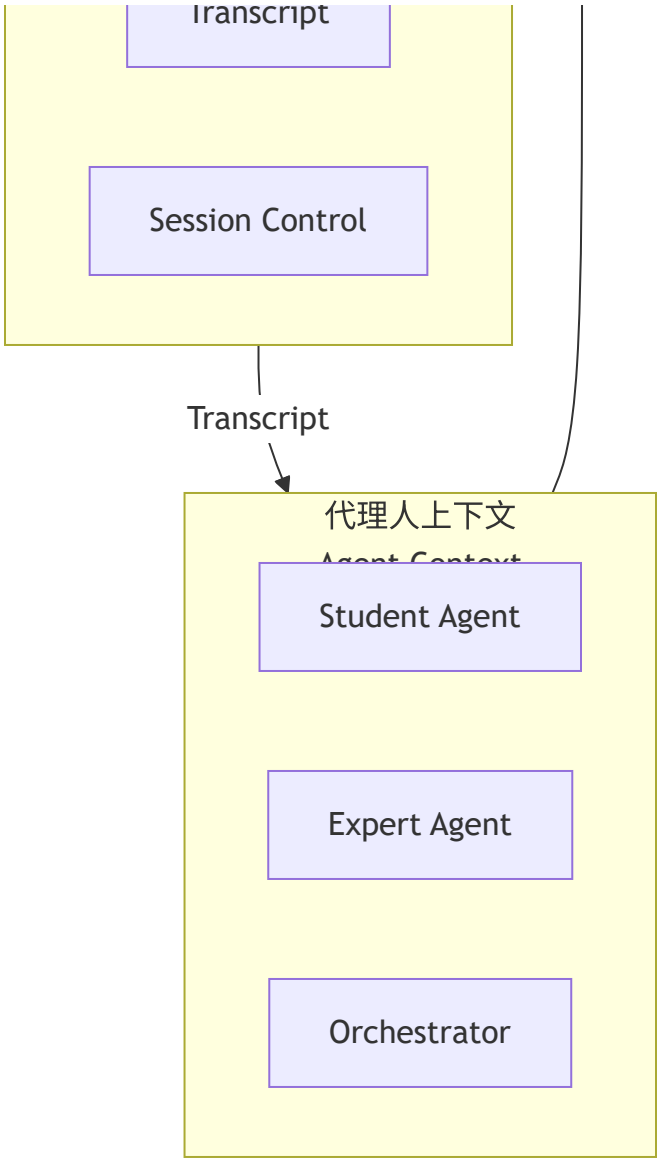
1.2 DDD 戰略設計 (Strategic DDD)

通用語言 (Ubiquitous Language)

術語	定義
試教會話 (Teaching Session)	一次完整的教學模擬過程，從開始到結束
虛擬學生 (Virtual Student)	AI 驅動的學生角色，模擬真實學生行為
專家教練 (Expert Coach)	AI 驅動的評估角色，依據教育理論提供回饋
教育方法論 (Pedagogical Methodology)	如 Satir 模式、SEL 框架等教學理論
逐字稿 (Transcript)	試教過程中所有對話的文字記錄
回饋儀表板 (Feedback Dashboard)	展示量化評估指標的視覺化介面
再練一次 (Practice Again)	結束回饋後立即重新開始試教的功能
場景協調器 (Scene Coordinator)	中央導演角色，控制對話節奏與發言順序
紙娃娃系統 (Paper Doll System)	透過切換 2D 靜態圖片呈現學生表情狀態

限界上下文 (Bounded Contexts)


評估回饋上下文
Evaluation Context
Metrics
Qualitative Feedback
Practice History
History
身份認證上下文
Identity Context
User
Authentication
Profile
User ID
Evaluation
試教會話上下文
Session Context
Teaching Session



上下文關係：

- **身份認證** → **試教會話**: 上游-下游關係，提供用戶身份
- **試教會話** → **代理人**: 試教會話為主導，代理人為協作者
- **代理人** → **評估回饋**: 生產者-消費者關係
- **評估回饋** → **身份認證**: 歷史記錄回寫用戶個人檔案

1.3 Clean Architecture 分層

```
┌──────────────────────────────────────────────────────────────┐
│                    Presentation Layer                        │
│  (React Web App, REST API Controllers, WebSocket Handlers)   │
├──────────────────────────────────────────────────────────────┤
│                    Application Layer                         │
│  (Use Cases: StartSession, PauseSession, GetFeedback, etc.) │
├──────────────────────────────────────────────────────────────┤
│                      Domain Layer                            │
│  (Entities: User, Session, Agent, Transcript, Evaluation)    │
│  (Services: OrchestratorService, EvaluationService)          │
├──────────────────────────────────────────────────────────────┤
│                   Infrastructure Layer                       │
│  (PostgreSQL Repo, Redis Cache, LLM Clients, STT Clients)    │
└──────────────────────────────────────────────────────────────┘
```

依賴規則:

- 內層不依賴外層
- Domain Layer 為純 Python，不依賴任何框架
- Infrastructure Layer 實現 Domain 定義的介面 (Repository Pattern)

1.4 技術選型與決策

分類	選用技術	版本
前端框架	React	18.x
前端語言	TypeScript	5.x
後端框架	FastAPI	0.100+
Agent 框架	LangChain	0.1.x
主資料庫	PostgreSQL	15+
向量資料庫	Chroma (本地) / Pinecone (生產)	-
快取	Redis	7.x
LLM (學生)	Qwen	-
LLM (專家)	OpenAI GPT-4o	-

分類	選用技術	版本
STT	OpenAI Whisper	-
TTS	(待定，可用 Edge TTS)	-

第 2 部分：需求摘要 (Requirements Summary)

2.1 功能性需求摘要 (Functional Requirements Summary)

ID	功能需求	優先級	對應 User Story
FR-1	用戶註冊與登入系統	P0 (MVP)	教師能建立帳號並登入系統
FR-2	試教會話控制（開始/暫停/結束）	P0 (MVP)	教師能控制試教流程
FR-3	虛擬學生語音互動	P0 (MVP)	教師能與 AI 學生進行語音對話
FR-4	專家評估與量化儀表板	P0 (MVP)	教師能獲得教學表現評分
FR-5	質性文字回饋（逐字稿分析）	P0 (MVP)	教師能獲得具體改進建議
FR-6	「再練一次」功能	P0 (MVP)	教師能立即重新練習
FR-7	歷史記錄查詢	P0 (MVP)	教師能追蹤過往表現
FR-8	學生未獲回應時再次提問	P1	增加互動擬真度
FR-9	暫停思考功能	P1	教師能在互動中暫停思考如何回應
FR-10	個人備忘錄功能	P2 (Post-MVP)	教師能記錄個人反思

2.2 非功能性需求 (Non-Functional Requirements - NFRs)

NFR 分類	具體需求描述	衡量指標/目標值
性能 (Performance)	API 端點 P95 延遲	`< 500ms` (不含 LLM 回應)
	LLM 回應延遲	`< 5s` (可接受，配合 UI 緩衝動畫)
可擴展性 (Scalability)	初期同時在線用戶	`100 concurrent users`

NFR 分類	具體需求描述	衡量指標/目標值
可用性 (Availability)	系統可用性	`99.5%` (MVP 階段)
安全性 (Security)	數據傳輸加密	`TLS 1.2+`
	身份驗證	`JWT Token`
音訊品質	STT 準確率	`> 95%` (中文)

第 3 部分：高層次架構設計 (High-Level Architectural Design)

3.1 選定的架構模式 (Chosen Architectural Pattern)

模式: `模塊化單體 (Modular Monolith) + 事件驅動局部應用`

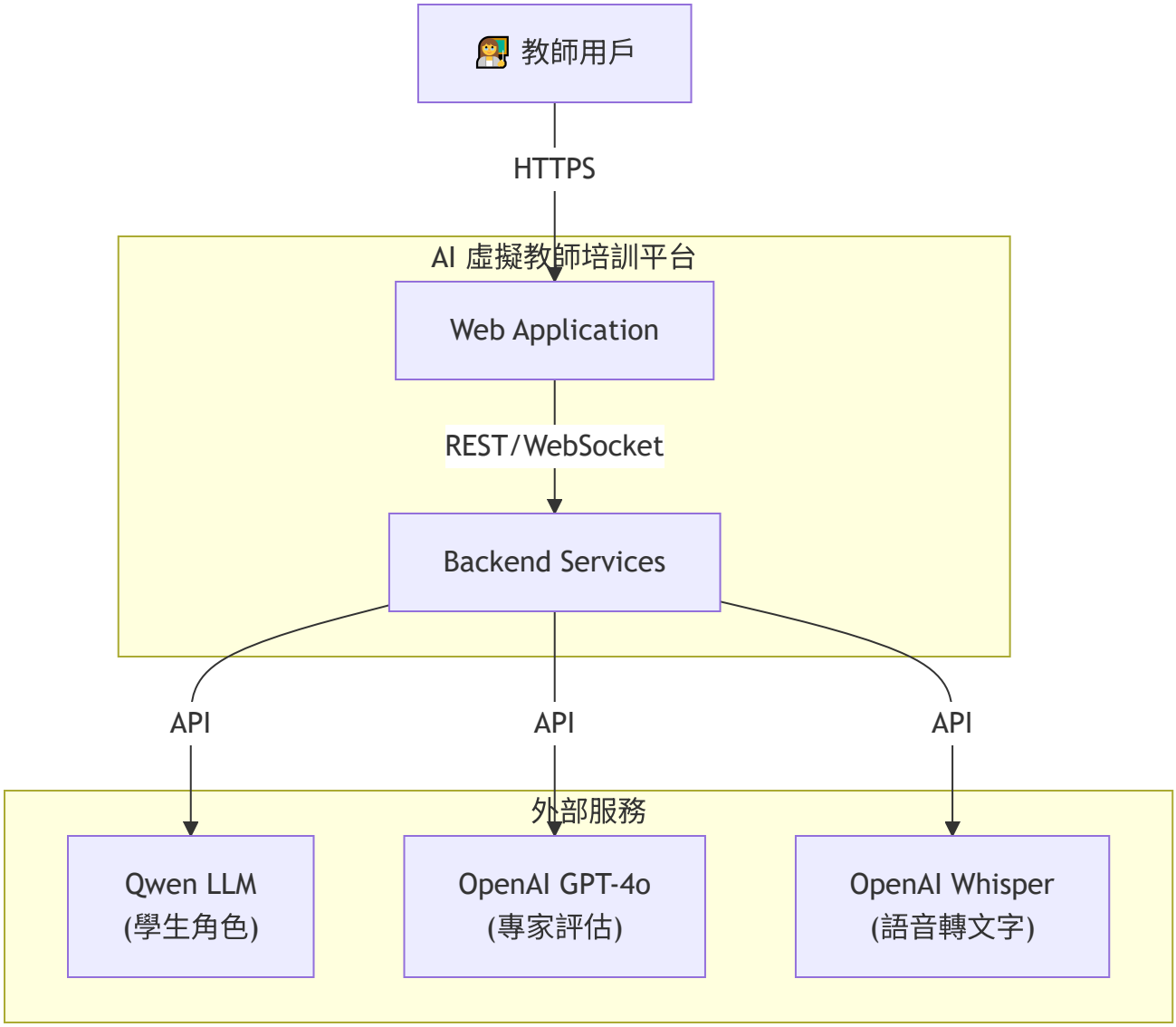
選擇理由:

1. **團隊規模適中**: 7 人團隊，微服務會增加不必要的運維複雜度
2. **快速迭代需求**: MVP 階段需要快速交付，單體部署更簡單
3. **清晰的模塊邊界**: 使用 DDD 限界上下文保持模塊獨立性，未來可拆分
4. **成本考量**: 減少基礎設施開銷

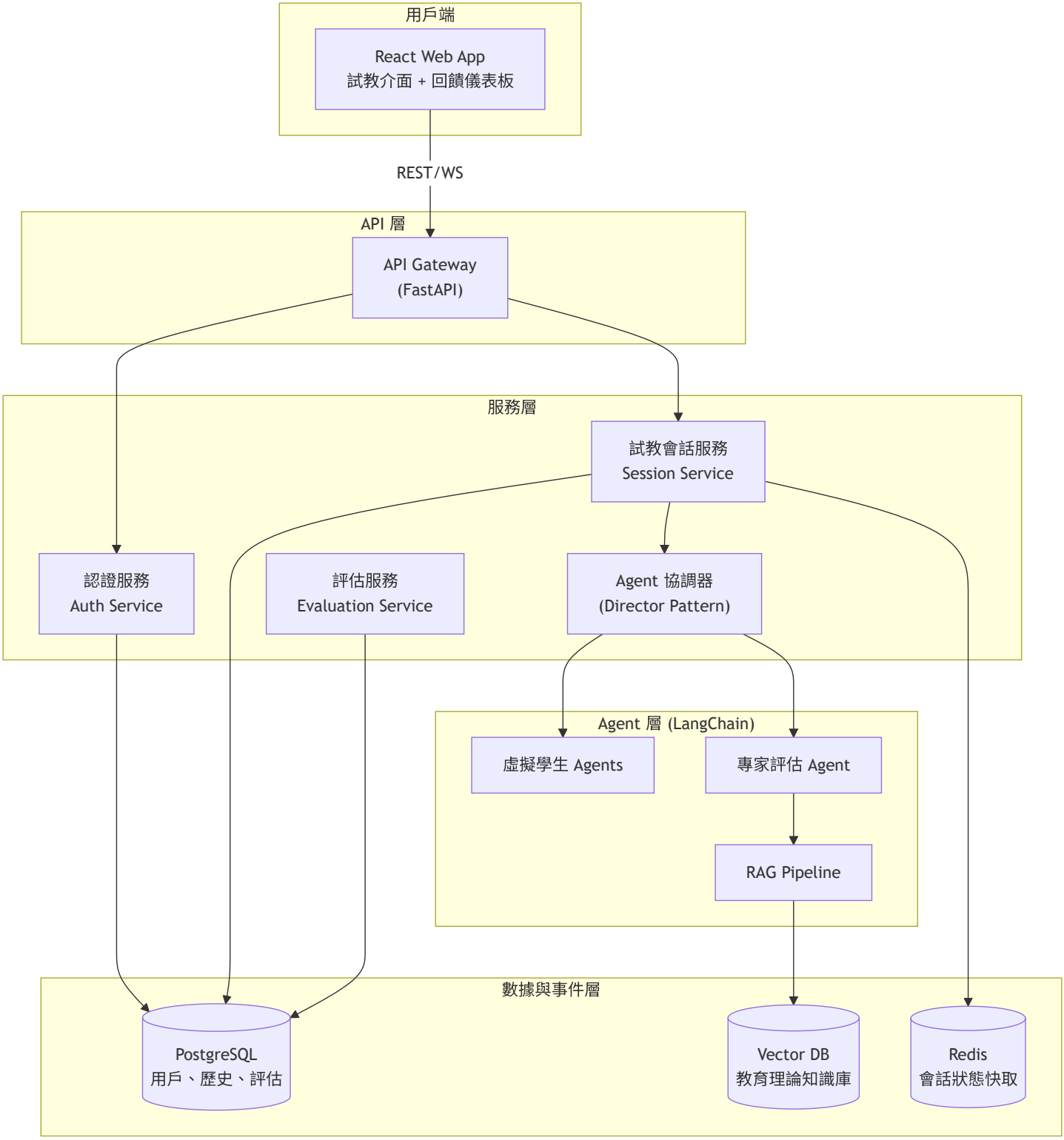
與微服務的權衡:

- 微服務適合大團隊、高可擴展需求
- 當前階段不需要獨立部署各模塊
- 保留未來遷移到微服務的可能性

3.2 系統上下文圖 (System Context Diagram)

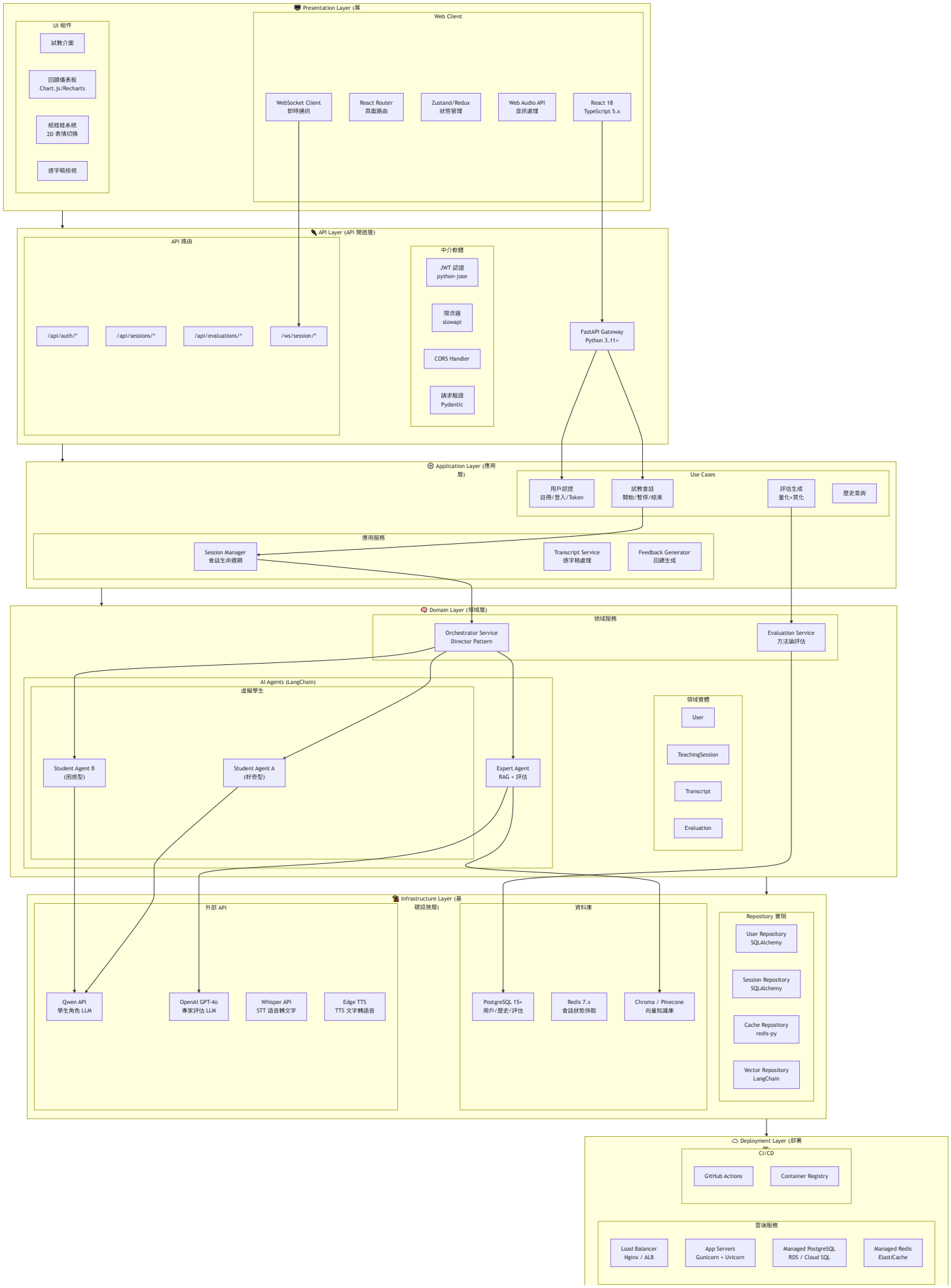


3.3 系統組件圖 (System Component Diagram)



3.4 分層式技術堆疊架構圖 (Layered Technology Stack Architecture)

此圖展示系統完整的技術堆疊分層架構，從基礎設施層一路往上到展示層：


Presentation Layer (展
UI 組件
試教介面
回饋儀表板
Chart.js/Recharts
紙娃娃系統
2D 表情切換
逐字稿檢視
Web Client
WebSocket Client
即時通訊
React Router
頁面路由
Zustand/Redux
狀態管理
Web Audio API
音訊處理
React 18
TypeScript 5.x
API Layer (API 閘道層)
API 路由
/api/auth/*
/api/sessions/*
/api/evaluations/*
/ws/session/*
中介軟體
JWT 認證
python-jose
限流器
slowapi
CORS Handler
請求驗證
Pydantic
FastAPI Gateway
Python 3.11+
Application Layer (應用層)
Use Cases
用戶認證
註冊/登入/Token
試教會話
開始/暫停/結束
評估生成
量化+質化
歷史查詢
應用服務
Session Manager
會話生命週期
Transcript Service
逐字稿處理
Feedback Generator
回饋生成
Domain Layer (領域層)
領域服務
Orchestrator Service
Director Pattern
Evaluation Service
方法論評估
AI Agents (LangChain)
虛擬學生
Student Agent B
(困惑型)
Student Agent A
(好奇型)
Expert Agent
RAG + 評估
領域實體
User
TeachingSession
Transcript
Evaluation
Infrastructure Layer (基礎設施層)
外部 API
Qwen API
學生角色 LLM
OpenAI GPT-4o
專家評估 LLM
Whisper API
STT 語音轉文字
Edge TTS
TTS 文字轉語音
資料庫
PostgreSQL 15+
用戶/歷史/評估
Redis 7.x
會話狀態快取
Chroma / Pinecone
向量知識庫
Repository 實現
User Repository
SQLAlchemy
Session Repository
SQLAlchemy
Cache Repository
redis-py
Vector Repository
LangChain
Deployment Layer (部署層)
CI/CD
GitHub Actions
Container Registry
雲端服務
Load Balancer
Nginx / ALB
App Servers
Gunicorn + Uvicorn
Managed PostgreSQL
RDS / Cloud SQL
Managed Redis
ElastiCache

完整技術堆疊分層說明：

層級	名稱	職責	核心技術
Presentation	展示層	用戶介面、視覺化、即時互動	React 18, TypeScript, WebSocket, Chart.js
API	API 閘道層	請求路由、認證授權、限流、驗證	FastAPI, JWT, Pydantic, CORS
Application	應用層	業務用例編排、會話管理、服務協調	Python Use Cases, Service Classes
Domain	領域層	核心業務邏輯、AI Agent 協調、評估演算法	LangChain, Director Pattern, RAG
Infrastructure	基礎設施層	資料持久化、外部 API 整合、快取	PostgreSQL, Redis, Chroma, LLM APIs
Deployment	部署層	容器化、雲端部署、CI/CD	Docker, GitHub Actions, AWS/GCP

層級依賴規則：

```
┌──────────────────────────────────────────────────────────────────┐
│  Presentation Layer (React)                                      │
│  ↓ depends on                                                    │
├──────────────────────────────────────────────────────────────────┤
│  API Layer (FastAPI Gateway)                                     │
│  ↓ depends on                                                    │
├──────────────────────────────────────────────────────────────────┤
│  Application Layer (Use Cases)                                   │
│  ↓ depends on                                                    │
├──────────────────────────────────────────────────────────────────┤
│  Domain Layer (Entities, Services, Agents)  ← 核心，不依賴外層    │
│  ↓ depends on (透過介面抽象)                                       │
├──────────────────────────────────────────────────────────────────┤
│  Infrastructure Layer (DB, APIs, Cache)                          │
│  ↓ deployed on                                                   │
├──────────────────────────────────────────────────────────────────┤
│  Deployment Layer (Docker, Cloud, CI/CD)                         │
└──────────────────────────────────────────────────────────────────┘
```

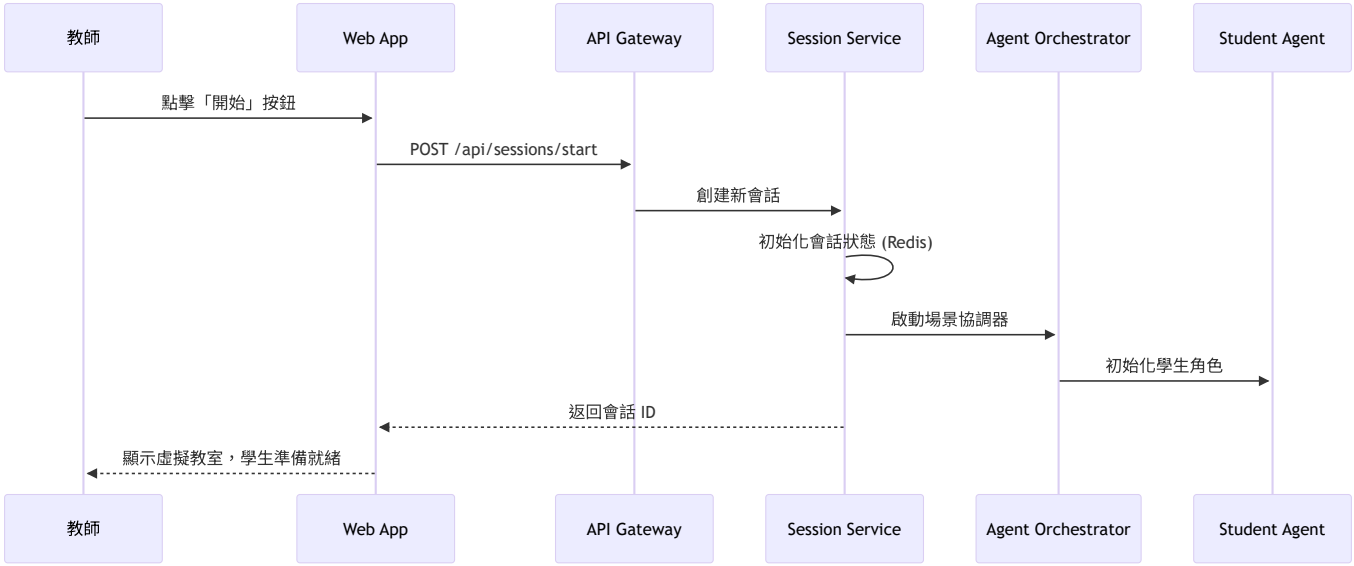
3.5 主要組件/服務職責 (Key Components/Services Responsibilities)

組件/服務名稱	核心職責	主要技術/框架	依賴
React Web App	試教介面、語音輸入、回饋視覺化、紙娃娃系統	React, TypeScript, Web Audio API	API Gateway
API Gateway	請求路由、JWT 認證、限流	FastAPI, python-jose	Auth Service
認證服務	用戶註冊、登入、Token 管理	FastAPI, bcrypt	PostgreSQL
試教會話服務	會話生命週期管理、狀態控制	FastAPI, Redis	PostgreSQL, Redis, Agent Orchestrator
Agent 協調器	中央導演模式、發言權管理、對話節奏控制	LangChain, Python	Student Agents, Expert Agent
虛擬學生 Agents	模擬學生回應、角色扮演	LangChain Agents	Qwen API

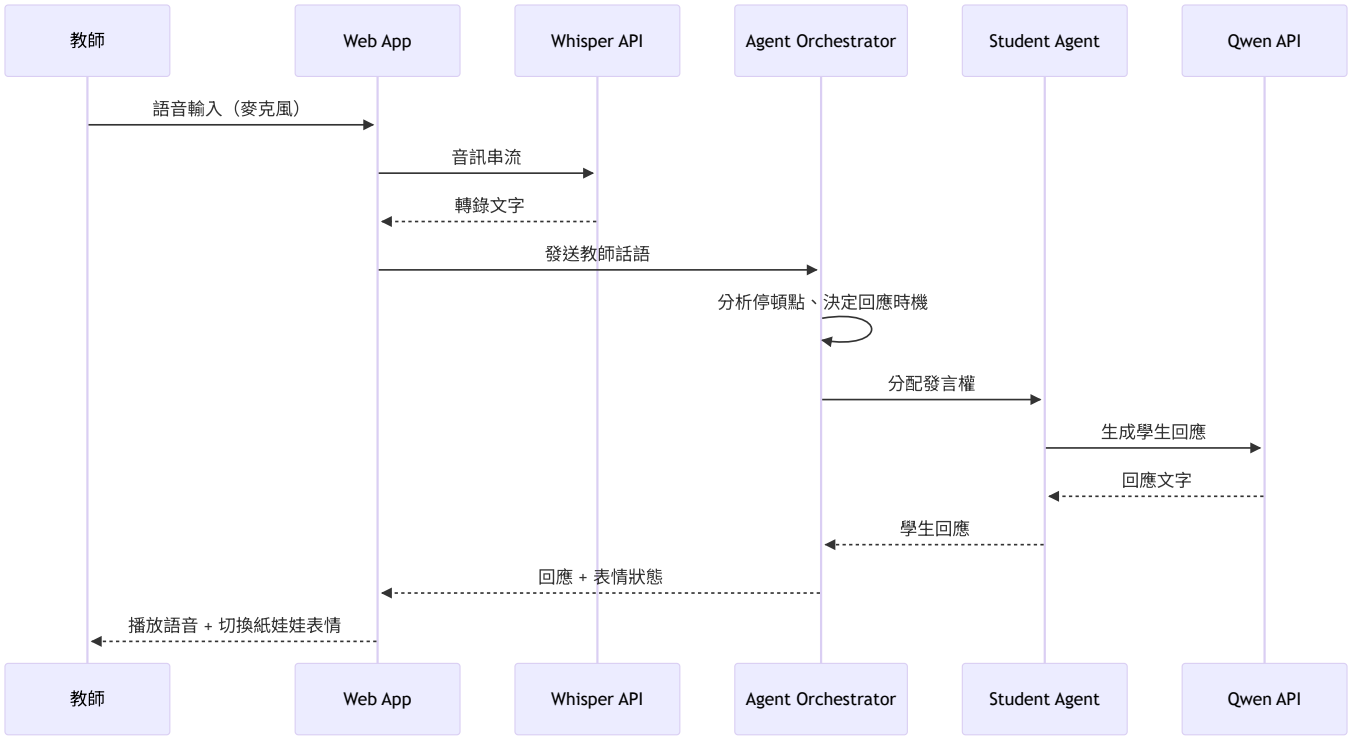
組件/服務名稱	核心職責	主要技術/框架	依賴
專家評估 Agent	RAG 評估、方法論分析、回饋生成	LangChain, RAG	GPT-4o API, Vector DB
評估服務	儀表板數據聚合、歷史記錄	FastAPI	PostgreSQL

3.5 關鍵用戶旅程與組件交互 (Key User Journeys)

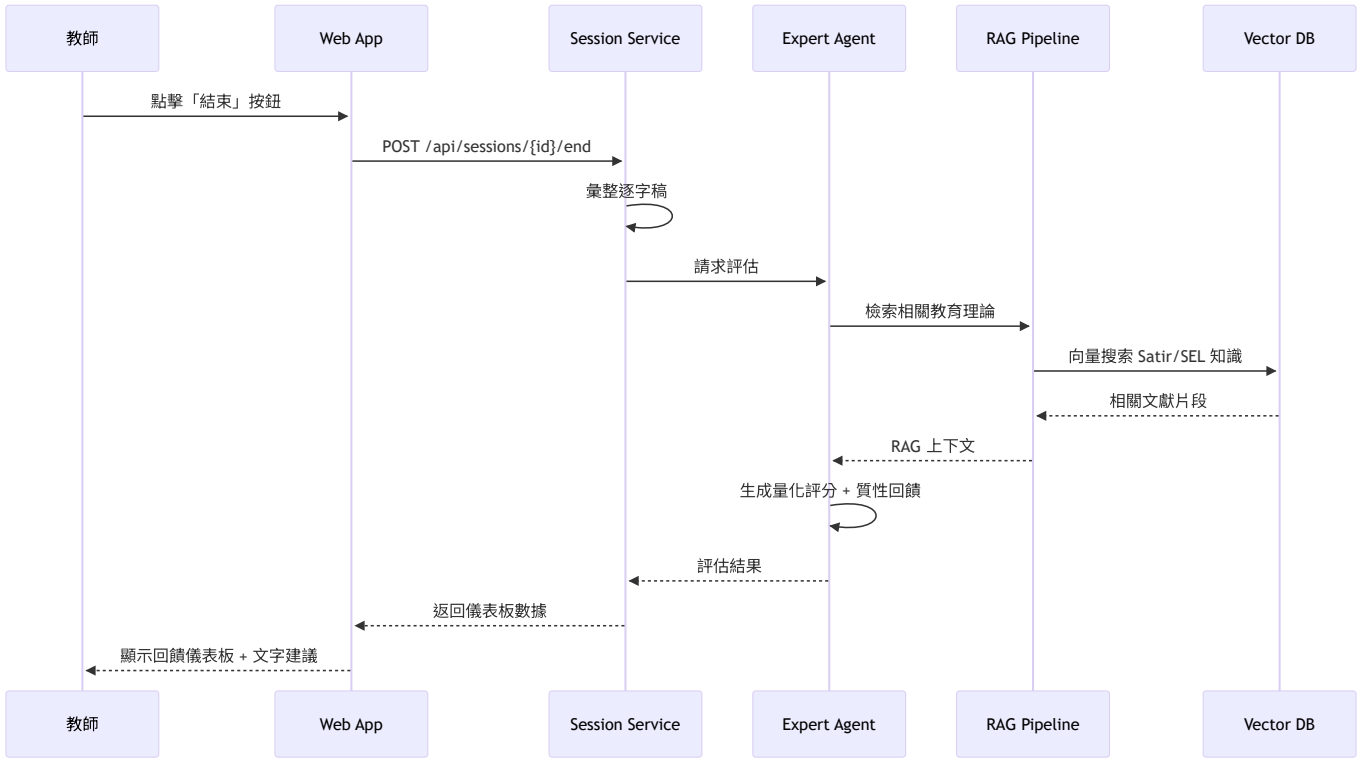
場景 1: 教師開始試教



場景 2: 語音互動循環



場景 3: 試教結束與評估



第 4 部分：技術選型詳述 (Technology Stack Details)

4.1 技術選型原則 (Technology Selection Principles)

1. **團隊熟悉度優先**: 選擇團隊已有經驗的技術棧
2. **開源與社群支持**: 優先選擇活躍的開源專案
3. **MVP 速度**: 選擇能快速開發的工具
4. **成本效益**: 平衡功能與 API 調用成本

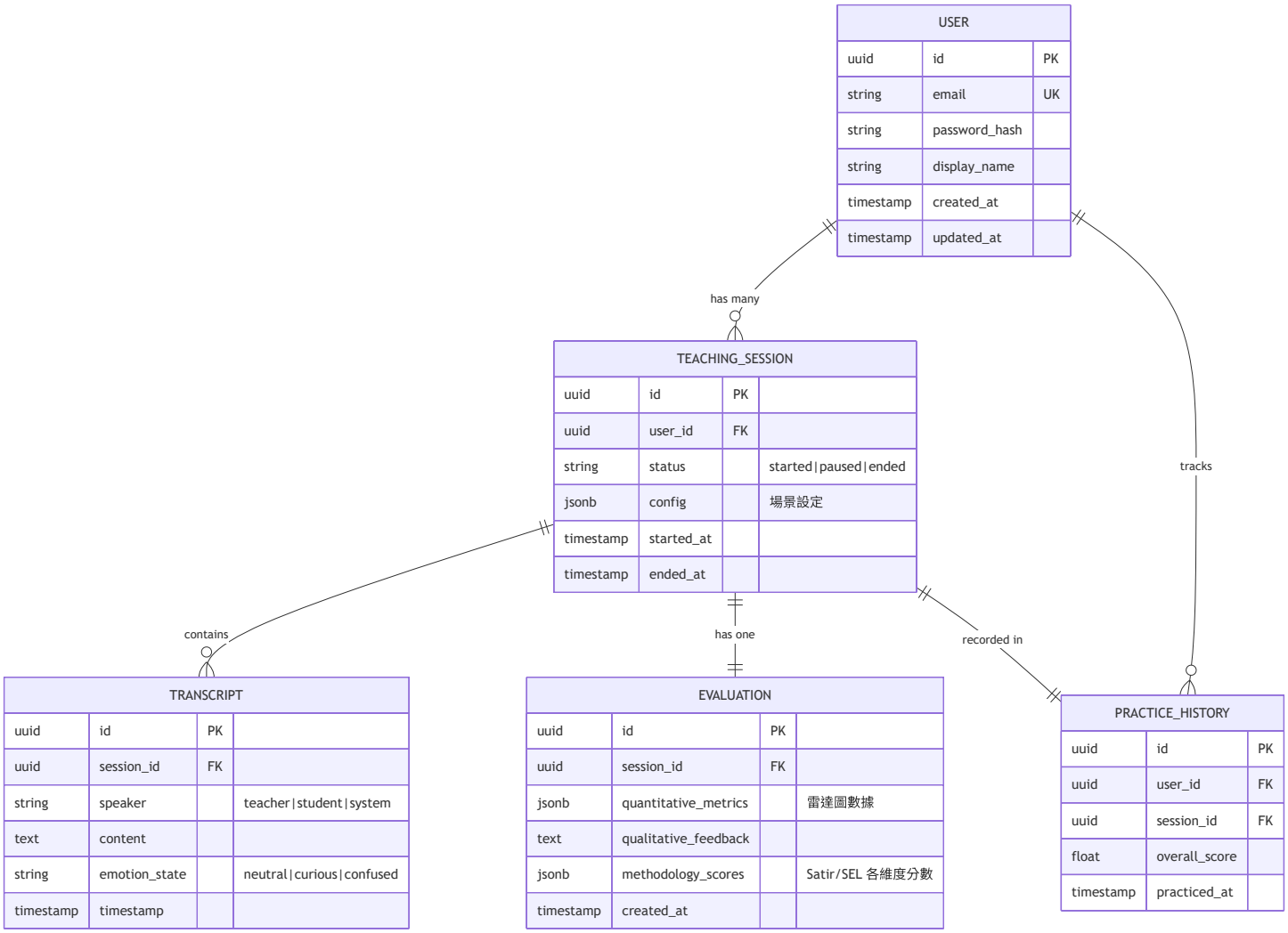
4.2 技術棧詳情 (Technology Stack Breakdown)

分類	選用技術	選擇理由	考量的備選方案	風險/成熟度
前端框架	React 18 + TypeScript	組件化開發、豐富生態系、團隊熟悉	Vue.js (文件提及)：功能相近但團隊傾向 React	成熟
後端框架	FastAPI	高性能異步、自動 OpenAPI 文檔、Python 生態整合	Flask：簡單但異步支持不佳；Django：偏重但對此場景過度	成熟
Agent 框架	LangChain	Multi-Agent 支持、RAG 整合、活躍社群	AutoGen (文件提及)：微軟方案，概念相似	成熟
主資料庫	PostgreSQL	事務支持、JSONB 靈活性、穩定可靠	MySQL：功能相近但 JSONB 支持較弱	成熟
向量資料庫	Chroma (開發) / Pinecone (生產)	Chroma 本地易用、Pinecone 託管省運維	Weaviate, Milvus：功能強大但運維複雜	成熟
快取	Redis	會話狀態管理、高性能、廣泛支持	Memcached：不支持複雜資料結構	成熟
LLM (學生)	Qwen	成本效益、支援中文、速度較快	Gemini Flash (文件提及)：替代選項	成熟
LLM (專家)	OpenAI GPT-4o	推理能力強、適合複雜評估任務	Claude：替代選項，需評估成本	成熟

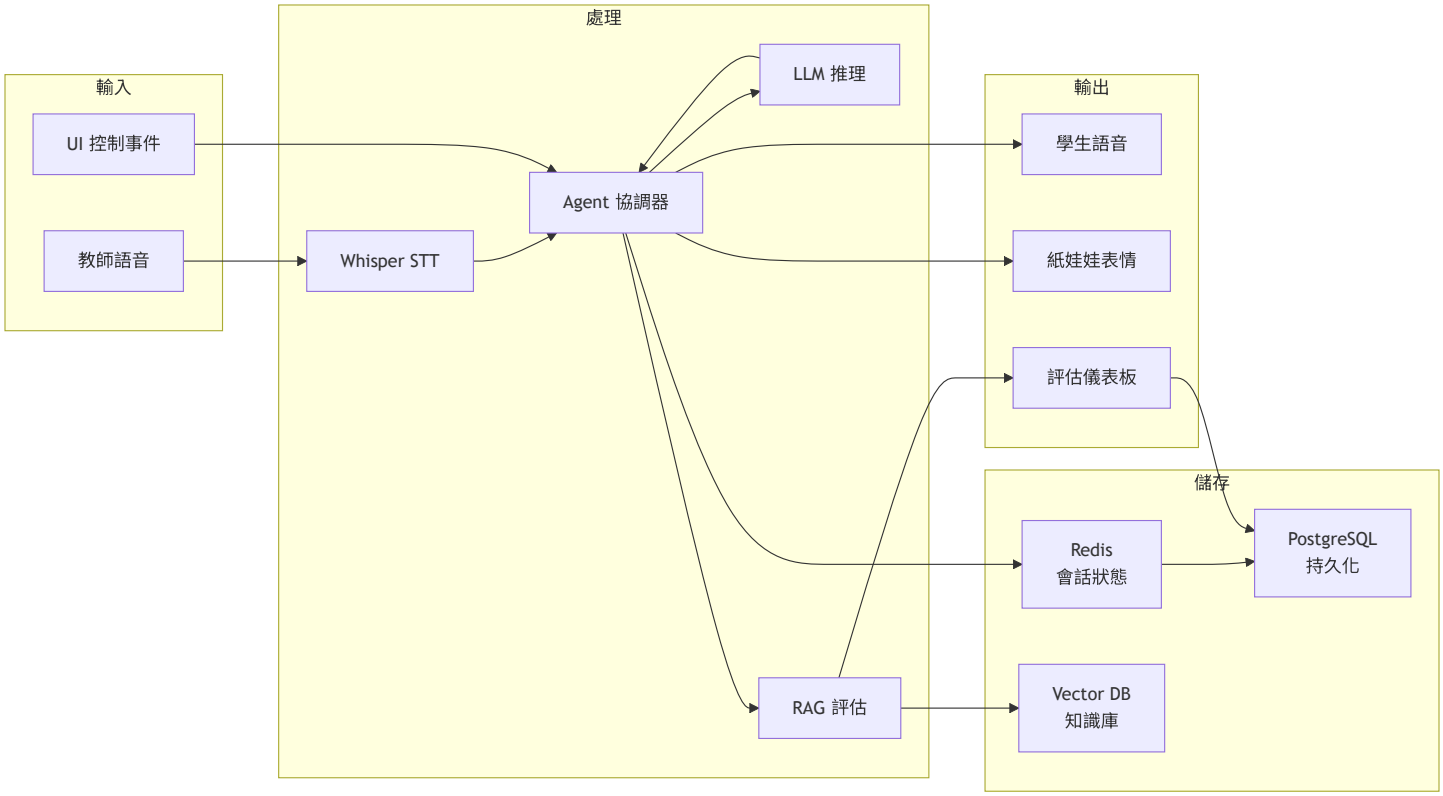
分類	選用技術	選擇理由	考量的備選方案	風險/成熟度
STT	OpenAI Whisper	中文支持佳、準確率高	Azure Speech：替代選項	成熟

第 5 部分：數據架構 (Data Architecture)

5.1 數據模型 (Data Models)



5.2 數據流圖 (Data Flow Diagrams)



5.3 數據一致性策略 (Data Consistency Strategy)

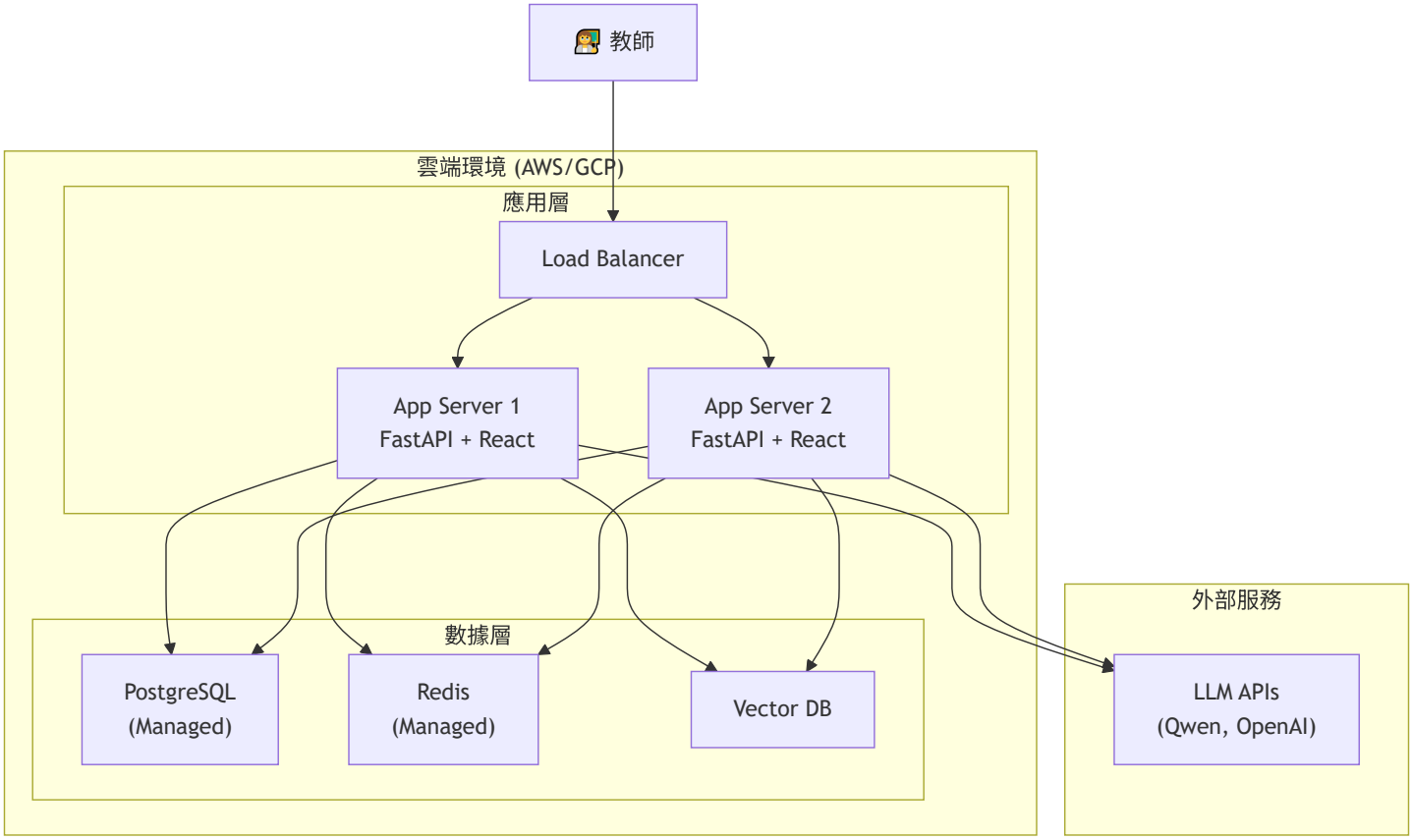
場景	一致性要求	策略
用戶認證資料	強一致性	PostgreSQL ACID 事務
試教會話狀態	最終一致性	Redis 快取 + 異步持久化
逐字稿記錄	最終一致性	批量寫入 PostgreSQL
評估結果	強一致性	直接寫入 PostgreSQL

5.4 數據生命週期與合規

- **數據分類**:
  - PII (個人識別資訊): email, display_name
  - 教學數據: transcripts, evaluations
- **數據加密**: 靜態加密 (PostgreSQL TDE)、傳輸中加密 (TLS)
- **數據保留**: 試教記錄保留 2 年，可由用戶主動刪除

第 6 部分：部署與基礎設施架構 (Deployment and Infrastructure)

6.1 部署視圖 (Deployment View)



6.2 CI/CD 流程 (CI/CD Pipeline)

6.3 環境策略 (Environment Strategy)

環境	用途	資源配置
Development	本地開發測試	Docker Compose, SQLite/PostgreSQL
Staging	整合測試、UAT	完整架構，小規模
Production	正式服務	高可用配置

第 7 部分：跨領域考量 (Cross-Cutting Concerns)

7.1 可觀測性 (Observability)

- **日誌 (Logging)**: 結構化 JSON 日誌，集中收集
- **指標 (Metrics)**: API 延遲、LLM 調用成功率、會話數量
- **告警 (Alerting)**: LLM API 錯誤率 > 5%、P95 延遲 > 10s

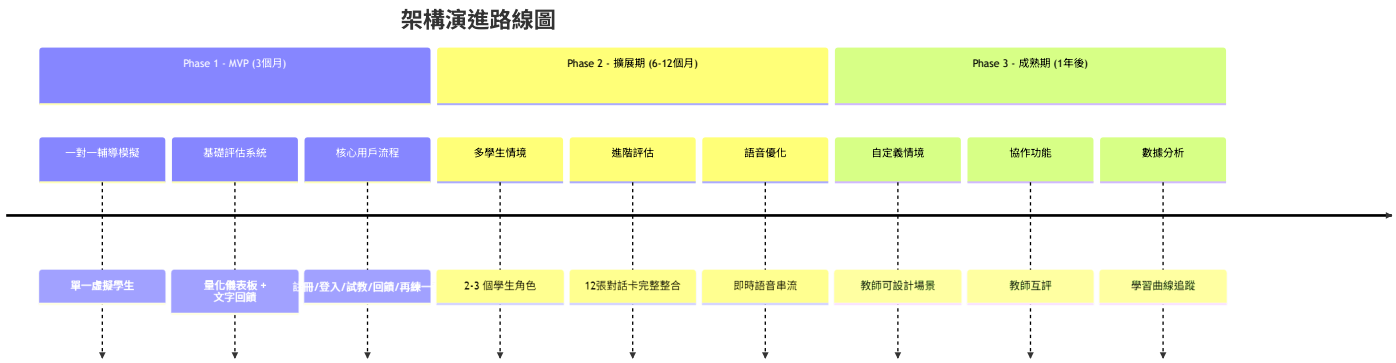
7.2 安全性與隱私 (Security and Privacy)

層面	策略
認證	JWT Token，有效期 24 小時
授權	用戶只能存取自己的數據
傳輸	TLS 1.2+
機密管理	環境變數 / Secret Manager
API Key 保護	後端調用 LLM API，前端不暴露

第 8 部分：風險與緩解策略 (Risks and Mitigation)

風險類別	風險描述	可能性	影響程度	緩解策略
技術風險	語音互動延遲影響體驗	高	中	UI 設計「學生思考中...」動畫緩衝；優先文字互動為主
技術風險	LLM 幻覺導致專家評估不準確	中	高	嚴格 RAG，強制引用知識庫原文；不使用純 LLM 生成
技術風險	多代理人對話混亂	中	中	中央導演模式嚴格控制發言權
依賴風險	LLM API 服務不穩定	低	高	準備備用 LLM 提供者 (Qwen ↔ Gemini)
成本風險	API 調用成本超支	中	中	使用較便宜的模型處理學生角色；設定使用量限制

第 9 部分：架構演進路線圖 (Architecture Evolution Roadmap)



第 10 部分：詳細設計 (Detailed Design)

10.1 MVP 與模組優先級 (MVP & Module Priority)

關鍵模組 (MVP Scope)

1. **用戶認證模組 (AuthModule)**
2. **試教會話模組 (SessionModule)**
3. **Agent 協調器模組 (OrchestratorModule)**
4. **評估回饋模組 (EvaluationModule)**

後續模組 (Post-MVP)

1. 多學生場景模組
2. 備忘錄模組
3. 進階數據分析模組

10.2 核心功能：模組設計

模組: AuthModule (認證模組)

- **職責**: 用戶註冊、登入、Token 管理
- **API 設計**:

方法	路徑	說明
POST	`/api/auth/register`	用戶註冊
POST	`/api/auth/login`	用戶登入，返回 JWT
GET	`/api/auth/me`	獲取當前用戶資訊
POST	`/api/auth/refresh`	刷新 Token

模組: SessionModule (試教會話模組)

- **職責**: 會話生命週期管理、狀態控制
- **API 設計**:

方法	路徑	說明
POST	`/api/sessions`	創建新會話
POST	`/api/sessions/{id}/start`	開始試教
POST	`/api/sessions/{id}/pause`	暫停試教
POST	`/api/sessions/{id}/resume`	繼續試教
POST	`/api/sessions/{id}/end`	結束試教
GET	`/api/sessions/{id}`	獲取會話詳情
GET	`/api/sessions`	獲取用戶所有會話

模組: OrchestratorModule (Agent 協調器模組)

- **職責**: 中央導演模式、發言權管理、對話節奏控制
- **WebSocket 事件**:

事件	方向	說明
`teacher_speech`	Client → Server	教師說話內容
`student_response`	Server → Client	學生回應 + 表情狀態
`student_thinking`	Server → Client	學生思考中狀態
`session_state`	Server → Client	會話狀態更新

模組: EvaluationModule (評估回饋模組)

- **職責**: RAG 評估、儀表板數據生成
- **API 設計**:

方法	路徑	說明
GET	`/api/evaluations/{session_id}`	獲取評估結果
GET	`/api/evaluations/{session_id}/transcript`	獲取逐字稿
GET	`/api/history`	獲取歷史評估列表

10.3 非功能性需求設計 (NFRs Design)

- **性能**: Redis 快取會話狀態，減少資料庫查詢
- **安全性**: 所有 API 透過 JWT 認證，用戶只能存取自己的數據
- **可擴展性**: 無狀態應用伺服器，可水平擴展

附錄 (Appendices)

A. 12張對話卡說明（模組化佔位）

> **注意**: 12張對話卡的詳細定義將作為獨立模組在後續迭代中加入。目前 MVP 階段使用簡化版評估維度。

預留維度:

1. 自我覺察
2. 溝通一致性
3. 同理心表達
4. 情緒調節
5. (其他待定...)

B. 紙娃娃系統狀態定義

狀態 ID	狀態名稱	圖片檔案	觸發條件
neutral	中性	`student_neutral.png`	預設狀態

狀態 ID	狀態名稱	圖片檔案	觸發條件
curious	好奇	`student_curious.png`	學生提問時
confused	困惑	`student_confused.png`	學生表示不理解時
thinking	思考中	`student_thinking.png`	等待教師回應
happy	開心	`student_happy.png`	學生理解時

文件審核記錄 (Review History):

日期	審核人	版本	變更摘要/主要反饋
2026-01-20	技術架構師	v1.0	初稿提交